

17-214/514: Agentic Software Development

What Is a Software System's Design?

Hammad Ahmad and Claire Le Goues • Wednesday, September 9, 2026 • Lecture

Administrivia

- If you are resubmitting your commit link for assignments, don't add it as a comment. Resubmit to Canvas, and we'll use your latest submission.
- We sent out AWS Academy invitations yesterday. Accept yours when you get a chance.
- The window for A1 verbal defenses ends on Monday.
- A2 is released. (More on this later.)

Two ways to build the same thing

A feature request for the booking service

The room booking service should be more helpful. If someone asks for a room at a time that's already taken, don't just reject it. Find them the next free hour and book that instead. Add tests, too.

Hit enter, go get coffee.

Agent returns...

```
public TimeInterval bookOrNextFree(String room, TimeInterval slot) {
    BookingResult result = bookings.book(room, slot);
    if (result == BookingResult.CONFIRMED) {
        return slot;
    }

    int length = slot.end() - slot.start();
    for (int start = slot.start() + 60; start + length <= DAY_END; start += 60) {
        TimeInterval candidate = new TimeInterval(start, start + length);
        if (BookingRules.canBook(DAY_START, DAY_END, repo.forRoom(room), candidate)) {
            bookings.book(room, candidate);
            return candidate;
        }
    }
    return null;
}
```

And it's green

tests	result	branch coverage
5	all passing	100%
Ship it?		

What does “the next free hour” mean?

Find them the next free hour and book that instead.

- The room is busy 9:00 to 10:15. You ask for 10:00. What should you get?
- You ask for ninety minutes. There is a one-hour gap. Now what?
- You ask for 8:00, before the building opens. Is that “taken”?
- Should it book the fallback, or offer it to you first?

Two of those it gets wrong

10:15, not 11:00

It steps forward an hour at a time. A room free at 10:15 is offered at 11:00.

Illegal is not taken

An 08:00 request is outside business hours. It is “helpfully” moved to 09:00.

These aren't "bugs", they're interpretations of what you meant.

Validation-forward, design-first, agentic

First: what does the sentence actually mean??

Question	Decision
Which direction?	Forward only, from the request
How far?	To 17:00, same day
What length?	The requested length, never shortened
Book, or propose?	Book, and return what was booked

Then: break it into checkable pieces

Task	Its check
The search, as a pure function	Eight cases, boundaries included
Wire it into the service	The returned slot is really held
The illegal cases	08:00 books nothing, moves nothing
The notification	One call, for the slot actually booked

Every check traces to a decision. No decision, nothing to check, no way to size the task.

"agent-sized tasks with validation"

```
## Task 1: the search, as a pure function
```

```
`nextFreeSlot(existing, request, dayEnd, step)` returns the first free slot  
of the same length at or after the request, or empty.
```

No clock, no repository, no notifier. It takes what it needs, returns a value.

****Check:**** eight cases. Requested slot free. Requested slot taken, next step free. Touching booking at the boundary (must not conflict). A gap too short for the request. A room free at 10:15 found at 10:15 and not 11:00.

****Fails before the code exists.**** Written first, red, committed red.

Written before the code. Committed red.

Nine checks, from those decisions

	passed	failed
What the agent built first	7 / 9	2
Built from the decisions	9 / 9	0

Evidence/artifacts

Vibed

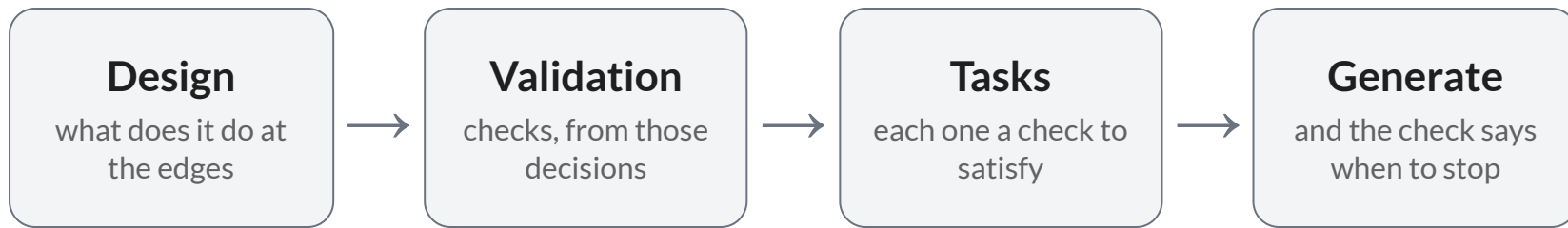
```
HelpfulBookingService.java  
HelpfulBookingServiceTest.java
```

Validation-first

```
SPEC.md  
PLAN.md  
SlotSearchTest.java    (initially committed red)  
SlotSearch.java  
ADR-001-fallback-search.md
```

We're asking for the right-hand-side to encourage the process.

The order is the point



- Skip the first and you have nothing to write checks about.
- Skip the second and “done” means whatever the agent says it means.

Checkpoint for A2: initial validation harness, and a version of a specification

What did we talk about last time?

- A spec says what correct means. Preconditions, postconditions, and invariants make it precise.
- Tests check the spec's claims. You can't test everything, but you can pick cases by equivalence class and probe the boundaries.
- Property-based testing: state a property, generate the inputs, shrink failures to a minimal case.
- A common approach to agentic SE: write the tests first, then generate code.
- Coverage reports what your tests ran, not how good the tests are.
- Mutation testing asks whether your tests would notice a change.

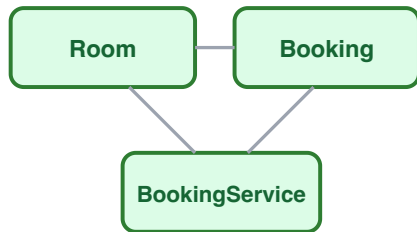
Learning goals for today

- Describe the major components of a software system's design
- Explain why invariants belong to the data model, not to individual operations
- Reason about structure at multiple scales, from a method to a subsystem
- Identify alternative decompositions of the same problem and weigh them
- Explain why a low representational gap matters more when an agent is writing the code

Both pass every test

Two agents, same prompt, same acceptance suite. Both green:

A: suite green



you can read the problem in the shape

B: suite also green

DoEverything.java (400 lines)

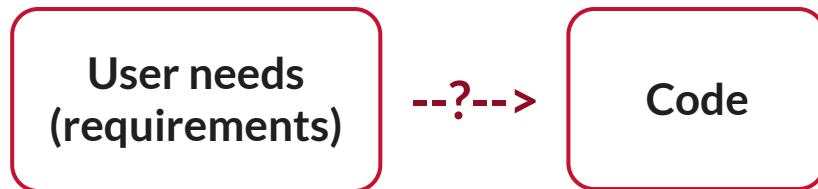
```
rooms[]: [17, 42, 17, 3, ...]
users[]: [ 2, 9, 2, 5, ...]
starts[]: [10, 11, 13, 9, ...]
(index 2 of all three arrays is one booking. Trust us.)
```

the reader has to keep track of what belongs together

Are they equally good?

From needs to code

How do you get from a need to working code?



An agent can cross that middle step in seconds. So, is your work done?

The questions a design has to answer

For the same need, you could write many different programs. Will yours be:

- Robust?
- Maintainable?
- Testable?
- Extensible?
- Scalable?
- Something-else-able?

Design involves choosing among those programs (and **quality attributes**).

So what is a design?

**The decisions about a system's
shape.**

What a design is **not**

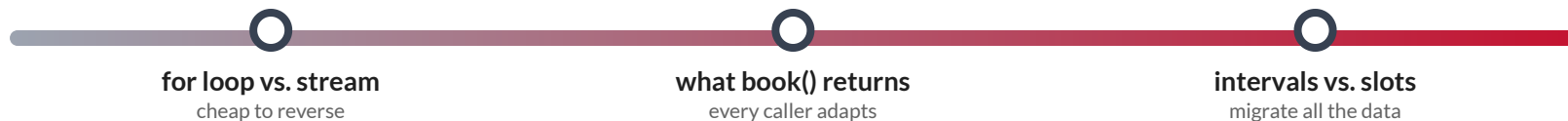
- Not the code. (The code is one **expression** of the design.)
- Not a diagram. (A diagram is a **picture** of part of it.)
- Not your folder layout. (That says where files live, not how the pieces depend on each other.)

A design is the set of decisions all of those are trying to express.

Decision or detail?

A test: how expensive is it to reverse?

- A detail is cheap to reverse.
- A decision can be expensive to reverse even with an agent. There may be data to **migrate** (rewrite what's already stored into the new shape) and callers to update.
- Agents make the details nearly free, so you can focus your judgment on the decisions.



<- details: the agent handles these freely

decisions: your judgment is important here ->

Three parts of a design

A design has three parts

WHAT IT KNOWS

Data

The data model: entities, relationships, state.

WHAT IT DOES

Operations

The APIs and behaviors: what you can ask it to do.

HOW IT'S ARRANGED

Structure

The modules: how data and operations are organized and who owns what.

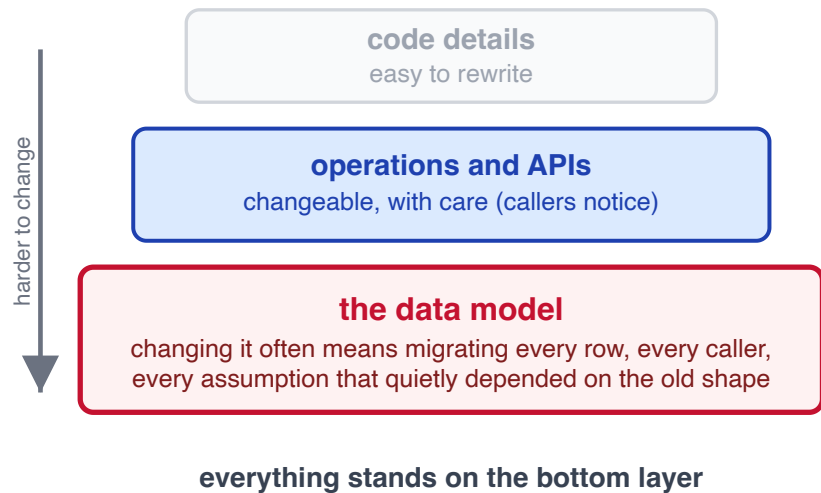
We'll spend weeks on each. Today: the whole picture.

Data: what the system knows

- The entities, their relationships, and the state the system holds onto.
- It's usually the most **durable** part of a design.
 - Behaviors come and go, but the data model is what everything else is built on.
- Start here. Nail down what the system knows and a lot of the operations design themselves.

Data is usually the hardest thing to change

- A function can just be rewritten/optimized.
- A data-model change is the expensive kind of reversal.
- So the data model gets your most careful design thinking and agent supervision.
- (More on this later, when data models need to scale.)



Operations: what the system does

- The behaviors you can invoke (the APIs, the methods, the verbs; more on this in a second).
- Each one is a little contract: give me this, I promise you that.
- This is where most of the code lives, and most of what an agent generates.

Have we reasoned about operations in this course before?

Structure: how it's arranged

- How data and operations get grouped into methods, classes, modules, subsystems.
- Where the boundaries are, and what crosses them.
- This is the part agents can be bad at...
 - ... and also the part that matters a bunch at scale.

Designing a booking system

Let's design a tiny room-booking system

The client's ask:

"The department has meeting rooms, and members should be able to reserve them. Each room has a name and a capacity. A member books a room for a time interval and can cancel a booking they made. Two bookings for the same room must never overlap, and anyone should be able to list a room's bookings to find a free time."

- Call out the nouns. Which ones would the system need to know about?
- Call out the verbs. Which ones are operations someone invokes?

Designing the data model

Read the client's ask again. Nouns are candidate concepts. *Verbs* are candidate operations.

"The department has meeting rooms, and members should be able to *reserve* them. Each room has a name and a capacity. A member *books* a room for a time interval and can *cancel* a booking they made. Two bookings for the same room must never *overlap*, and anyone should be able to *list* a room's bookings to find a free time."

- You can prune synonyms ("reserve" and "book" are the same operation) and scenery ("the department").
- You can also prune implementation words. The client never said database, JSON, etc.
- "Overlap" is a verb, but nobody invokes it. "Never overlap" is a **rule**.

Concept, attribute, or implementation detail?

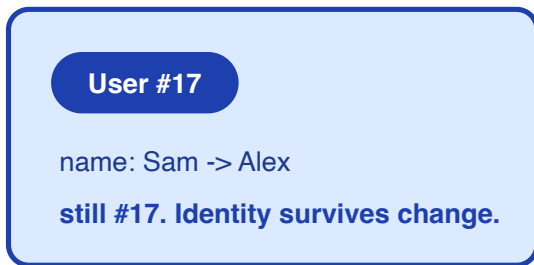
Ask yourself: would the client recognize it?

Candidate	Verdict	In the model?
Booking	A concept. The domain talks about it, and it has rules of its own.	Yes (entity)
capacity	An attribute of Room . In the real world it's just a number.	Yes (value)
Map<Room, List<Booking>>	Implementation. How you might store it, not what it is.	No

As an aside: entities and values

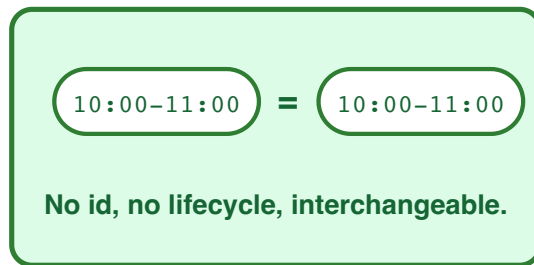
- An entity is a thing you track. It gets an id and a lifecycle.
- A value is just data, with no id. Any equal one is interchangeable.
- Which is which is a design call. If `Booking` were a value, you could not say **which** booking to cancel.

entity: has identity



in code: an id and a careful lifecycle
(you need to point at THAT one to cancel it)

value: equal means the same



record TimeRange(start, end)
...and you're done

The booking model

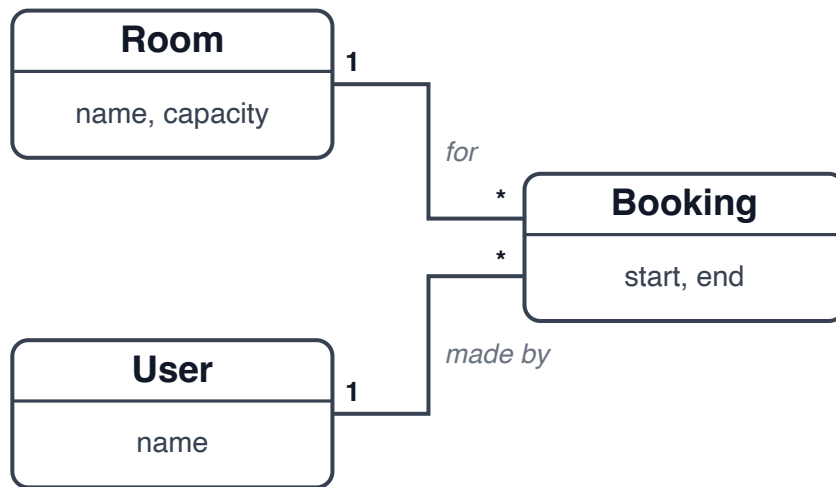
What survives the prune:

User	id, name
Room	id, name, capacity
Booking	id, user, room, start, end

- A **Room** has many **Booking**s. A **User** makes **Booking**s.
- A **Booking** is a room held for a time interval. (Already a design choice: is a booking a half-open interval `[start, end)`, or a pile of fixed slots?)
- The client ask never said `id`. But "cancel a booking they made" implies the ability to point at one specific booking.
 - Usually, identity is the first thing the design adds.

The same model as a picture

- Boxes are concepts. Lines are relationships. The numbers say "how many".
- Read a line in both directions. "A Room has many Booking s. Each Booking is for exactly one Room ."
- Deliberately informal (for now).



What does this picture commit you to?

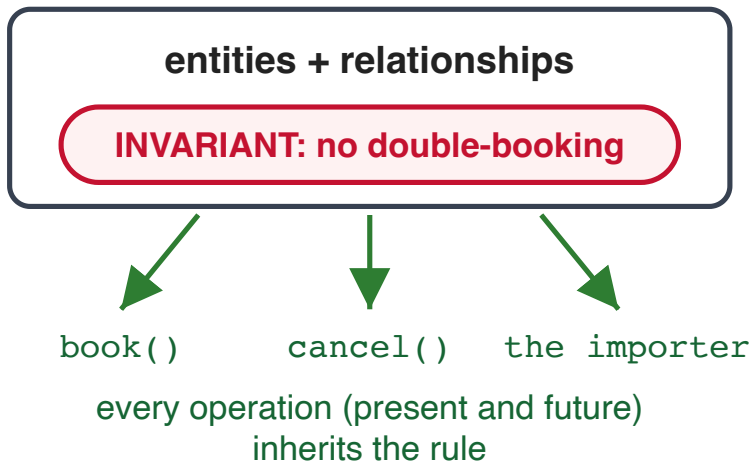
Read it like a critic.

- **What it claims:** every `Booking` belongs to exactly one `Room` and one `User`. Rooms outlive their bookings.
- **What it hides:** where does "no double-booking" live? Who creates a `Booking`?
- **What's expensive to change:**
 - Intervals vs. fixed slots is baked into `Booking`, and into every booking already stored.
 - "Exactly one `Room`" becomes a single `room` field everywhere. Letting a booking span rooms later means a migration.

The model has rules

"No room is double-booked." Which operation does that rule belong to?

in the data model



buried in book()'s docstring

```
/** books a room.  
 * (oh and never double-book) */  
Booking book(u, r, slot) ...
```

cancel()? next month's
agent-added op?

if your agent never sees it,
you/it will rediscover the invariant the hard way

Where the invariant is **enforced** is a separate decision.

Where does the reference live?

"A room has many bookings." Fine. Who holds whom?

- `Room` holds a `List<Booking>`? `Booking` points at its `Room`? Both (and now they can disagree)?
- Try skipping `Booking` entirely: `Map<Room, List<TimeRange>>`.
 - Compact! And `Booking` just became a value again.

Whichever you + your agent pick, every operation that follows inherits it.

Back to operations

```
addRoom(name, capacity)
book(user, room, start, end)
cancel(booking)
listBookings(room)
```

- Each is a contract with rules hiding inside.
- `book` quietly assumes the user exists, the room exists, the time is valid, and nothing already overlaps it.
 - Where do those checks live? (We'll see this later under "responsibility assignment".)

What should book() return?

- `void` ?
 - Then how does the caller cancel later?
 - An id? The whole `Booking` ?
- And when the slot is taken:
 - Throw an exception?
 - Return null? Return a result the caller must inspect?
- Every answer is a contract your callers live with. You do not want your agent to quietly pick/make up assumptions.

Where do the verbs come from?

Try to name your operations after what the **domain** does. (Agents are reasonably OK at this, by the way.)

For `Booking` :

- `createBooking`, `readBooking`, `updateBooking`, `deleteBooking` are the **database's** (CRUD) verbs, not the domain's.
- Does "update a booking" even exist in this domain? Or is it cancel-and-rebook, with different rules (and maybe a fee)?
- The operation list is itself a design decision. What you **refuse** to offer is design too.

Read the A2 contract the same way

`core-api-v1.md` is a rulebook you have to obey. It is also somebody's design.

```
POST /api/auth/register
 201  {"token": "...", "user": {"id": "...", "username": "..."}}
 409  username already taken
 400  missing or empty username or password
```

- Why 201 and not a plain 200? 201 tells the caller something exists now that did not before.
- Why hand back the user and a token instead of nothing? So the caller can act next without a second round trip.
- Can you leave something opaque in the contract?
 - If you promise nothing about their format, changing it later breaks no promise*.

What if a request is wrong in three ways at once?

Bad token, no such channel, empty body. Which failure does the caller hear about?



- The contract fixes one order, and every protected endpoint follows it.
- Another order would have been just as defensible. Picking one makes every system's answer predictable.
- Leave it unstated and each endpoint is free to answer differently.

How does a contract grow without breaking its clients?

The A2 contract gets revised all semester. What you build now has to survive that.

- **Tolerant reader.** Ignore fields you do not recognize, on both sides.
- **Reserved namespace.** Your extensions live on `x_` fields and `/api/x/...` routes, and the core promises never to touch those.

You will make the same decisions in your capability surfaces this week.

Back to structure. Where does it all go?

- One big `App` class that does everything?
 - (Please, no.)
- Separate `Rooms`, `Bookings` modules with clear owners?
- Layers: storage, domain logic, API surface?



Responsibilities and boundaries

Every design assigns responsibilities

- A responsibility is something the system must **know**, or something it must **do**.
- Design is largely deciding **who** gets each one.
- "A room can't be double-booked": whose job is enforcing that rule?

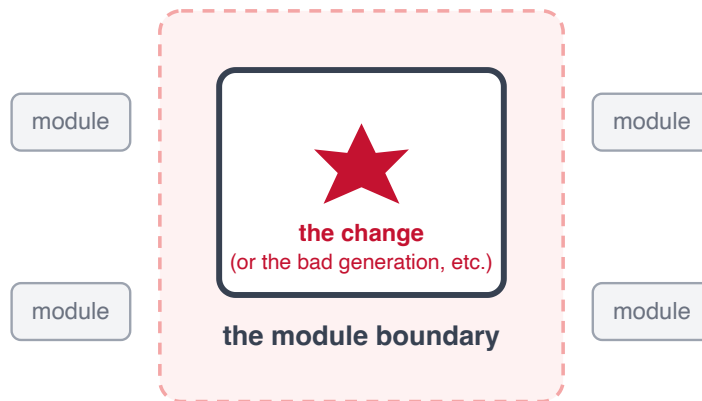
Where should that rule live?

- In the `Booking` constructor?
 - It barely knows what else is on the room.
- In `Room.book(...)` ?
 - The room knows its bookings. Promising.
- In a separate service that orchestrates the check?
 - Maybe, at scale.

Each of these results in a different design.

You can treat boundaries as promises

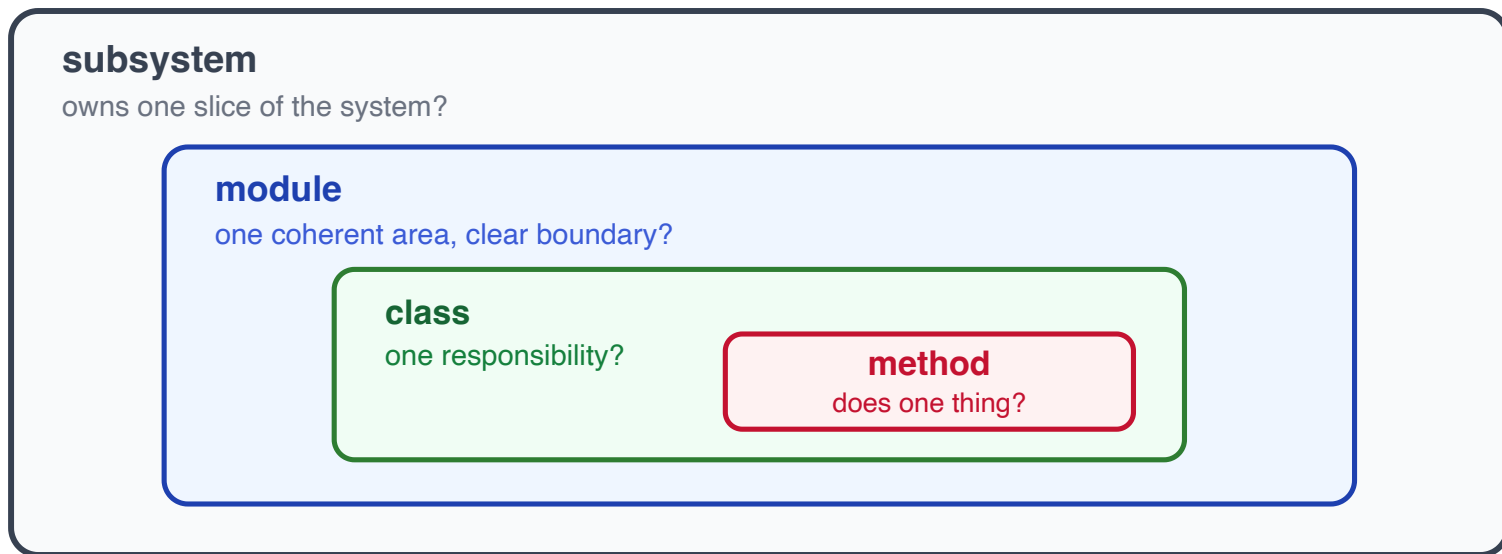
- A module's boundary says "**this** is mine, **that** is yours, talk to me only through these operations".
- Good boundaries should let you change the inside without breaking the outside.
- Bad boundaries often leak. Everyone reaches into everyone.



a boundary decides (among other things)
how far wrong can spread

Structure happens at every scale

Same questions, different zoom level

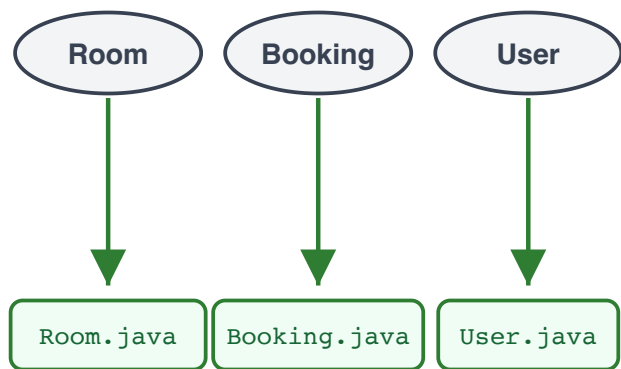


"What is this responsible for, and what's its boundary?"

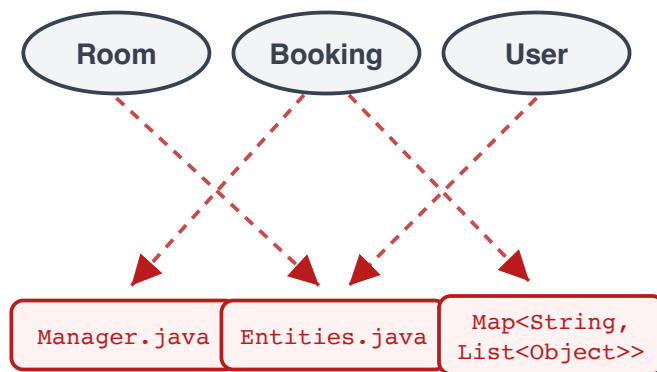
Representational gap

Good software looks like its problem

- A **low representational gap**: the concepts in the code mirror those in the domain.
- A high gap means every reader (agents included!) translates in their head.
 - And bugs can hide in this translation.



low gap: read the code, recognize the problem



high gap: every reader translates in their head

Naming *is* design

- The names you choose make the representational gap concrete.
- `Room`, `Booking`, `book()` tell the reader (and the agent) which world they're in.
- `Data`, `doStuff()` don't tell much.
- Naming appropriately is hard.
 - e.g., `interrupted()` vs. `isInterrupted()` for Java `Thread`.
- Agents can be good at naming when the domain's words are in context.
 - Context rot = annoying inconsistencies.



doStuff()

reserve()

Model the domain

- Don't shape the core around the UI ("it's whatever the form sends").
- Don't shape it around the database ("it's just rows").
- Shape it around the **problem**. The screen and the table bend to the model.

Why this matters **more** with an agent

- An agent reproduces whatever gap your design hands it.
- Clean, domain-shaped design? The agent stays oriented and fills in sensible local structure.
- Muddy, tangled design? The agent extends the muddle.

Alternatives and tradeoffs

There is no correct design

- But there are **many** incorrect ones.
- For our booking system, consider: storing a booking as a time interval and storing it as fixed slots.
 - Both are defensible!
- Choose deliberately, and know what you traded.

Two designs, same behavior. Which is better?

It depends.

- On how you'll **verify** it (e.g., slots enumerate cleanly, intervals bring boundary cases).
- On what's likely to **change** (recurring bookings? variable-length slots?).
- On what needs to be **reused** elsewhere.
- On who has to **understand** it later.

Another example: where does "availability" live?

OPTION 1

Compute it on the fly

From the bookings, on every ask. Simple, and always correct.

OPTION 2

Cache a free/busy calendar

Per room. Fast, but it can drift out of sync with the truth.

OPTION 3

Store open slots as records

Their own entities. Powerful, and heavier to keep consistent.

Quality attributes

- Each option for availability was better at *something* (simpler, faster, easier to keep consistent).
- **Quality attributes**: performance, modifiability, testability, reliability, simplicity (the "-ilities").
- Use them as vocabulary for arguing that one design beats another.
- Heads up: No design wins on every attribute. A choice trades some for others.

Can you use an agent for the design itself?

- Sure!
 - Ask for a first draft of the data model.
 - Ask for, say, three candidate decompositions and what each one makes expensive.
- Then audit the drafts. For example:
 - Did the agent invent concepts the requirements never had?
 - What does the picture hide?
 - What's expensive to change?
- The verdict is yours. Choosing needs context the agent may not have (what's likely to change, who maintains it, what you're willing to pay, etc.).

If you pin down only one thing...

...make it the data model.

- Most durable, hardest to reverse, and everything else stands on it. The highest leverage per line of spec.
- Let the agent draft operations **against** a model you chose.
- (Later, we will turn this dial up to "at scale." It will be fun.)

Wait, why design **after** verification?

- Weeks 1-2: verification, because it's the bottleneck on trusting generated code.
- But good design is **what makes software verifiable** in the first place: testable, modular, observable.
- Good design makes verification cheap(er).

You made six decisions today. Where do they live?

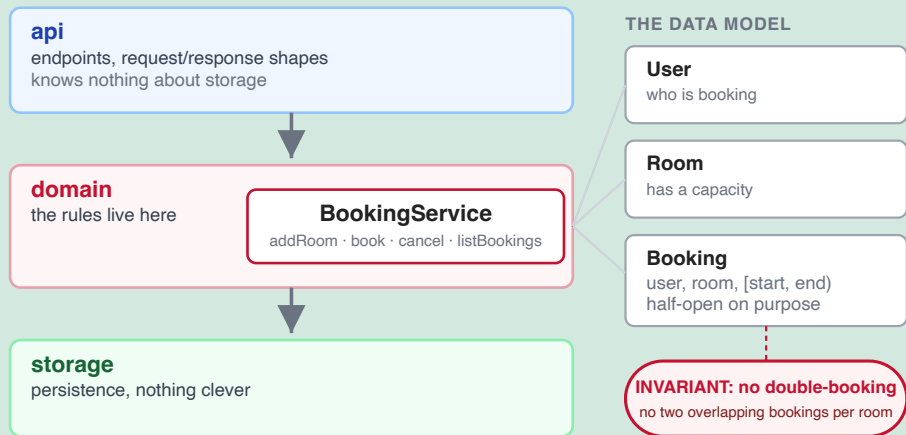
- Intervals vs. slots. Where availability lives. Who holds the reference. What `book()` returns. Where the rule is enforced. The module layout.
- Later, someone ask: "wait, why intervals?"
 - You/your team doesn't remember. The agent's context is also probably long gone.
- An unwritten decision gets made again later.
 - Maybe differently.
 - Or by someone (or some agent) who never knew it was settled.

Next time: how to write down decisions so they stay decided.

Let's practice: re-slice it

We structured the booking system as layers: storage / domain / API. Your turn:

- Propose a **different** decomposition. (By feature? Around the calendar? Something weirder?)
- For your alternative: name one thing that got easier and one that got harder.
- Which of the two would you rather hand to an agent, and why?



Assignment 2 is live

- Greenfield design: **your** data model, **your** API, **your** storage choice.
- The checkpoint next Tuesday asks for your design on paper. `CAPABILITY.md`, plus a harness that validates it and the core contract.
- You'll live with your decisions, since A3, A5 and A6 build on this system.
 - This does not mean every decision is *final*. You can change your mind. Is it free/cheap? You'll find out. :-)

Summary

- A design is the durable shape of a system (data, operations, structure), not its syntax.
- Design is assigning responsibilities and drawing boundaries, at every scale from method to subsystem.
- A low representational gap (code that looks like its problem) matters **more** with agents, because they amplify whatever you hand them.
- There's no correct design, but many wrong ones. Choose deliberately, and know what you traded. (It depends. On what?)
- The design decisions stay yours, and getting them right makes generation tractable.

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
--------------	----	--------------	----	-------------------	----	--------------	----	--------------	-----------	-----	---------------	-----	---------------

- **Friday, Lab 3:** design critique. Read a provided design, diagnose its problems in writing, and propose something better.
- **Monday:** communicating and recording designs.
- **Tuesday next week:** A2 checkpoint, 11:59pm. `CAPABILITY.md` and a harness covering it and the core contract.